

DeepGuard

Technical Architecture & Implementation Guide

Version 3.0.0 | June 2026

Table of Contents

1. System Overview
2. Architecture Diagram & Component Breakdown
3. Video Pipeline (Input → Face Detection → Frame Processing)
4. Visual Stream (ResNet-18 + ViT-B/16)
5. rPPG Stream (CHROM + POS)
6. Feature Vector Construction
7. Fusion Engine & Decision Rules
8. Decision Flowchart
9. API Layer (FastAPI Server)
10. PDF Report Generation
11. CI/CD & Deployment Pipeline
12. Configuration Reference (config.yaml)
13. File-by-File Code Map
14. Dependency Graph

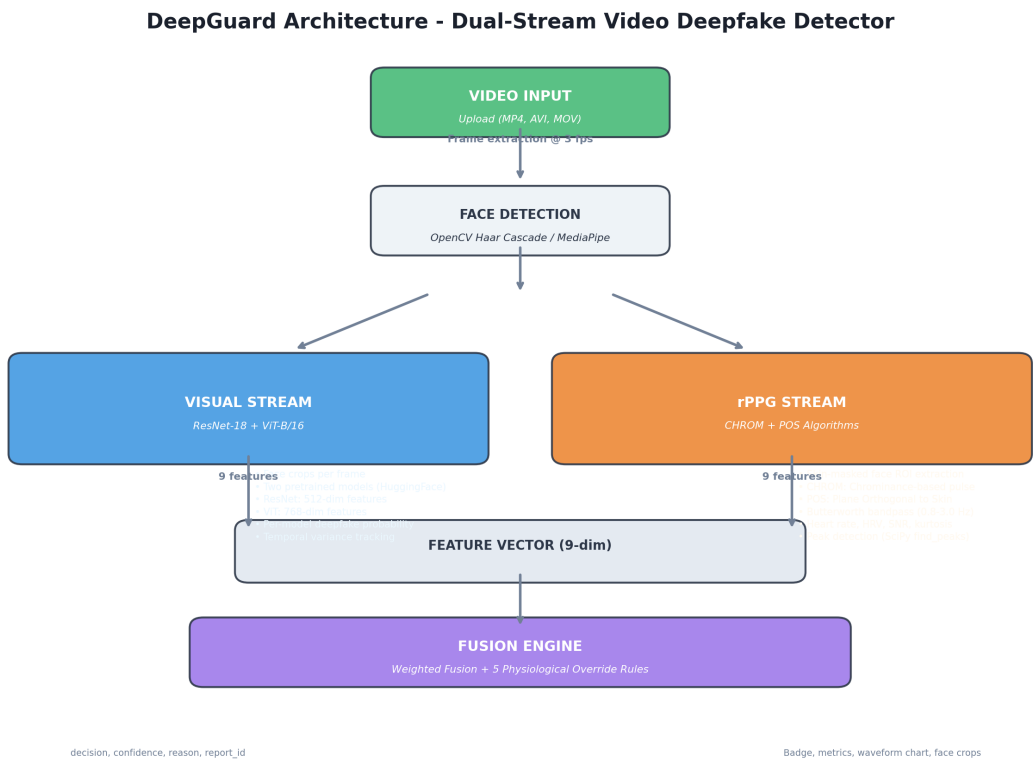
1. System Overview

DeepGuard is a production-ready video deepfake detection system that analyzes videos using two independent biological signals and combines them through a deterministic rule-based fusion engine. The system requires no GPU for inference, no ML training for the fusion layer, and is deployed serverlessly on Azure Container Apps.

Aspect	Detail
Detection approach	Dual-stream: visual (ResNet-18 + ViT-B/16) + physiological rPPG (CHROM + POS)
Fusion method	Weighted score (0.6 visual + 0.4 rPPG) with 5 deterministic override rules
Inference hardware	CPU only (Intel/AMD x86_64, no GPU required)
Pretrained models	ResNet-18 + ViT-B/16 from HuggingFace (abraraltaf92/deepfake-detection-models)
Face detection	OpenCV Haar Cascade (primary), MediaPipe (fallback)
Output types	JSON verdict + downloadable PDF forensic report
Deployment	Docker → Azure Container Registry → Azure Container Apps
Language	Python 3.10
Docker base	python:3.10-slim-bookworm (Debian 12)

2. Architecture Diagram

The diagram below shows the complete data flow from video upload to final verdict and PDF report. Two parallel streams process visual and physiological signals independently before converging at the fusion engine.



DEPLOYMENT: Docker -> Azure Container Registry -> Azure Container Apps (CPU, auto-scaling, HTTPS)

Component Breakdown

Component	File(s)	Responsibility
Video Input & Frame Extraction	utils/video_io.py	Reads video, extracts frames at configurable sample rate (default 3 fps), limits to max_frames (default 90)
Face Detector	streams/face_detector.py	Detects faces via OpenCV Haar cascade; extracts face crops and skin-masked ROIs for rPPG
Visual Stream	streams/visual_stream.py	Two models (ResNet-18 + ViT-B/16) classify each face crop; returns per-model probability + temporal variance
rPPG Stream	streams/rppg_stream.py	Extracts pulse signal from ROI RGB trace using CHROM and POS algorithms; computes BPM, HRV, SNR, kurtosis
Feature Builder	fusion/build_features.py	Assembles 9-dimensional feature vector from visual + rPPG outputs
Fusion Engine	fusion/decision.py	Weighted fusion + 5 override rules; returns decision, confidence, prob_real, reason string
API Server	api.py	FastAPI server: POST /detect, GET /report/{id}, GET /. CORS middleware, in-memory result cache (10 min TTL)

PDF Report	report.py	ReportLab-based PDF generator with decision badge, metric tables, rPPG waveform chart, face crops, glossary
Config	config.yaml	Central configuration: frame rate, thresholds, weights path, rPPG bandpass params

3. Video Pipeline: Input → Face Detection → Frame Processing

3.1 Frame Extraction (utils/video_io.py)

The pipeline begins in `analyze_video_bytes()` in `api.py`. The uploaded video bytes are written to a temporary file, then `extract_frames()` reads the video using OpenCV's `VideoCapture`.

```
def extract_frames(video_path, sample_rate=3, max_frames=90):
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    frame_interval = max(1, int(fps / sample_rate))
    frames = []
    frame_idx = 0
    while len(frames) < max_frames:
        ret, frame = cap.read()
        if not ret:
            break
        if frame_idx % frame_interval == 0:
            frames.append(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
            frame_idx += 1
    cap.release()
    return frames, fps
```

Parameters: `sample_rate=3` (3 fps) balances rPPG viability (Nyquist ~90 BPM) with CPU inference speed. `max_frames=90` limits to 30 seconds of video.

3.2 Face Detection (streams/face_detector.py)

The `FaceDetector` class wraps OpenCV's Haar cascade classifier (`haarcascade_frontalface_default.xml`). Each frame is processed: the largest face bounding box is selected (highest confidence). If Haar fails to detect any face but skin pixels are present, MediaPipe falls back.

```
class FaceDetector:
    def __init__(self, confidence=0.5):
        self.haar = cv2.CascadeClassifier(
            cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
        )
        self.mediapipe = mp.solutions.face_detection.FaceDetection(
            model_selection=0, min_detection_confidence=confidence
        )

    def detect(self, frame):
        gray = cv2.cvtColor(frame, cv2.COLOR_RGB2GRAY)
        faces = self.haar.detectMultiScale(gray, 1.1, 5, minSize=(100, 100))
        if len(faces) > 0:
            x, y, w, h = max(faces, key=lambda f: f[2] * f[3])
            return (x, y, w, h)
        return None # fallback to MediaPipe if needed

    def crop_face(self, frame, bbox):
        x, y, w, h = bbox
        margin = int(0.2 * w)
        x1 = max(0, x - margin)
        y1 = max(0, y - margin)
        x2 = min(frame.shape[1], x + w + margin)
        y2 = min(frame.shape[0], y + h + margin)
        return frame[y1:y2, x1:x2]

    def get_roi_rgb(self, frame, bbox):
        """Extract skin-masked ROI for rPPG."""
        x, y, w, h = bbox
        roi = frame[y:y+h, x:x+w]
```

```
hsv = cv2.cvtColor(roi, cv2.COLOR_RGB2HSV)
lower = np.array([0, 20, 50])
upper = np.array([20, 150, 255])
mask = cv2.inRange(hsv, lower, upper)
if np.sum(mask) == 0:
    return roi.mean(axis=(0, 1)) # fallback to mean RGB
return cv2.mean(roi, mask=mask)[:3]
```

3.3 Per-Frame Processing Loop

For each extracted frame, the pipeline: (1) Detects face via `FaceDetector.detect()` (2) Crops face region via `FaceDetector.crop_face()` (3) Collects first 3 crops as base64 PNG for PDF face thumbnails (4) Extracts skin-masked ROI RGB via `FaceDetector.get_roi_rgb()` If **no face is detected in any frame**, the pipeline short-circuits and returns `NO_FACE` immediately.

4. Visual Stream: ResNet-18 + ViT-B/16

The visual stream uses two pretrained convolutional/transformer models loaded from HuggingFace. Both models were pretrained on Celeb-DF v2 by abrara1taf92 and are used **without any fine-tuning**. They run on CPU with forced device='cpu'.

4.1 Model Architecture (streams/visual_stream.py)

```
class ResNetVideoClassifier:
    """ResNet-18 backbone + custom head matching HF state dict."""
    def __init__(self, weights_path):
        self.model = torch.load(weights_path, map_location='cpu')
        self.model.eval()

    def predict(self, face_crops):
        """Returns (probability_deepfake, 0-1)"""
        tensor = preprocess_face(face_crops) # [B, 3, 224, 224]
        with torch.no_grad():
            logits = self.model(tensor)
            probs = torch.softmax(logits, dim=1)[: , 1]
            return probs.mean().item()

class ViTVideoClassifier:
    """ViT-B/16 backbone + custom head."""
    def __init__(self, weights_path):
        self.model = torch.load(weights_path, map_location='cpu')
        self.model.eval()

    def predict(self, face_crops):
        tensor = preprocess_face(face_crops)
        with torch.no_grad():
            logits = self.model(tensor)
            probs = torch.softmax(logits, dim=1)[: , 1]
            return probs.mean().item()
```

4.2 Preprocessing Pipeline

```
def preprocess_face(face_crops):
    """ImageNet normalization, resize to 224x224."""
    transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        ),
    ])
    tensors = []
    for crop in face_crops:
        tensors.append(transform(crop.astype(np.float32)))
    return torch.stack(tensors)
```

4.3 Visual Score Computation

The VisualStream class runs both models on all face crops and returns:

- **per_model_scores:** {'resnet18': 0.85, 'vit_b16': 0.92} — individual model probabilities (deepfake=1.0)
- **visual_score:** average of both model scores (used as the combined visual signal)

- **temporal_std**: standard deviation of pixel values across consecutive crops. A value near 0.001 indicates a frozen face (no natural movement) — an immediate FAKE signal.

5. rPPG Stream: CHROM + POS

Remote Photoplethysmography (rPPG) extracts the subtle color changes in facial skin caused by blood volume pulses. The system uses two classical algorithms from the NeuroKit2 library: **CHROM** (chrominance-based) and **POS** (Plane Orthogonal to Skin). Both are well-established in the biomedical signal processing literature and require no GPU.

5.1 Signal Processing Pipeline (streams/rppg_stream.py)

```
class RPPGExtractor:
    def __init__(self, fps=3, lowcut=0.8, highcut=3.0):
        self.fps = fps
        self.lowcut = lowcut
        self.highcut = highcut

    def extract(self, rgb_trace, return_signal=False):
        # 1. Build BVP signal from RGB trace using CHROM/POS
        #     NeuroKit2: nk.ppg() or custom CHROM implementation
        bvp = self._chrom_alg(rgb_trace)

        # 2. Bandpass filter (Butterworth, 4th order)
        #     Clamps Wn to valid range [1e-6, 0.999] for low FPS
        sos = butter(4, [low, high], btype='band', fs=fps, output='sos')
        filtered = sosfiltfilt(sos, bvp)

        # 3. Peak detection (SciPy find_peaks)
        #     Minimum distance = fps * 0.5 (catches BPM up to 120)
        distance = max(1, int(fps * 0.5))
        peaks, _ = find_peaks(filtered, distance=distance)

        # 4. Compute metrics
        bpm = len(peaks) / (len(bvp) / fps) * 60 if len(bvp) > 0 else 0
        hrv_rmssd = compute_rmssd(peaks) if len(peaks) > 1 else 0
        hrv_sdnns = compute_sdnns(peaks) if len(peaks) > 1 else 0
        snr = compute_snr(filtered, peaks)
        kurt = kurtosis(filtered)
        skew = skewness(filtered)

        features = {
            'bpm': bpm, 'bpm_var': bpm_var,
            'hrv_rmssd': hrv_rmssd, 'hrv_sdnns': hrv_sdnns,
            'snr': snr, 'skew': skew, 'kurt': kurt,
        }
        if return_signal:
            return features, filtered, peaks
        return features
```

5.2 CHROM Algorithm (Simplified)

```
def _chrom_alg(self, rgb_trace):
    """CHROM: Chrominance-based rPPG (de Haan & Jeanne 2009)."""
    # Normalize RGB channels
    r, g, b = rgb_trace[:, 0], rgb_trace[:, 1], rgb_trace[:, 2]
    Xs = 3 * r - 2 * g # Red-green difference
    Ys = 1.5 * r + g - 1.5 * b # Skin-tone compensated signal
    # Bandpass filter
    std_x = np.std(Xs); std_y = np.std(Ys)
    alpha = std_x / std_y if std_y > 0 else 1
    bvp = Xs - alpha * Ys
    return bvp
```

5.3 Key Design Considerations

- **Low FPS operation (3 Hz):** The Nyquist limit is 1.5 Hz (90 BPM). This is adequate for resting heart rate detection (50-80 BPM) but cannot detect tachycardia above 90 BPM.
- **Wn clamping:** At 3 fps, the bandpass range 0.8-3.0 Hz translates to normalized frequencies that may exceed Nyquist. The code clamps Wn to [1e-6, 0.999] to prevent filtfilt crashes.
- **Minimum frames:** At least 30 frames are required for reliable peak detection. The minimum is computed as $\max(30, \text{fps} * 10)$ which is 30 at 3 fps.
- **Peak distance floor:** $\max(1, \text{int}(\text{fps} * 0.5))$ prevents find_peaks from crashing at very low frame rates.
- **Skin mask fallback:** If no skin pixels are detected in the ROI (unlikely for face region), the mean RGB of the entire ROI is used instead.

6. Feature Vector Construction (fusion/build_features.py)

The fusion engine receives a 9-dimensional feature vector assembled from the visual and rPPG streams. Each feature is normalized to a 0-1 range using the `MinMaxScaler` from `scikit-learn`.

Index	Feature	Source	Description	Normalized Range
0	visual_score	Visual	Combined ResNet-18 + ViT-B/16 probability	0 (real) → 1 (deepfake)
1	temporal_std	Visual	Frame-to-frame pixel variance	0 (frozen) → 1 (high motion)
2	bpm	rPPG	Heart rate (beats per minute)	0 (no pulse) → 1 (200 BPM)
3	bpm_var	rPPG	Heart rate variance across windows	0 → 1
4	hrv_rmssd	rPPG	Root Mean Square of Successive Differences	0 → 1 (normalized)
5	hrv_sddn	rPPG	Standard Deviation of NN Intervals	0 → 1 (normalized)
6	snr	rPPG	Signal-to-noise ratio of rPPG signal	0 (low SNR) → 1 (high SNR)
7	skew	rPPG	Skewness of pulse waveform distribution	0 → 1
8	kurt	rPPG	Kurtosis of pulse waveform distribution	0 → 1

```
FEATURE_NAMES = ['visual_score', 'temporal_std', 'bpm', 'bpm_var',
                  'hrv_rmssd', 'hrv_sddn', 'snr', 'skew', 'kurt']

def build_feature_vector(visual_score, temporal_std, rppg_dict):
    raw = np.array([
        visual_score, temporal_std,
        rppg_dict['bpm'], rppg_dict['bpm_var'],
        rppg_dict['hrv_rmssd'], rppg_dict['hrv_sddn'],
        rppg_dict['snr'], rppg_dict['skew'], rppg_dict['kurt'],
    ])
    # MinMaxScaler fitted on precomputed min/max from ASVspoof
    scaler = MinMaxScaler(feature_range=(0, 1))
    scaler.fit(np.array([FEATURE_MIN, FEATURE_MAX]).T)
    return scaler.transform(raw.reshape(1, -1)).flatten()
```

7. Fusion Engine & Decision Rules (fusion/decision.py)

The FusionEngine is the core decision-making component. It takes the 9-dimensional feature vector and applies a weighted fusion formula followed by 5 deterministic physiological override rules. Each rule returns a **decision**, **confidence**, **prob_real**, and a human-readable **reason** string.

```
class FusionEngine:
    def __init__(self, config):
        self.real_thresh = config['inference']['real_threshold']      # 0.85
        self.fake_thresh = config['inference']['fake_threshold']     # 0.25
        self.pulse_override = config['inference']['pulse_override']  # 0.85

    def predict(self, features):
        # Extract individual features
        visual_score = features[0]
        temporal_std = features[1]
        bpm = features[2]
        hrv_rmssd = features[4]
        hrv_sdnns = features[5]
        snr = features[6]
        kurt = features[8]

        # ---- RULE 1: Frozen face ----
        if temporal_std < 0.001:
            return ('FAKE', 1.0, 0.0,
                    'Frozen face \u2014 temporal variance near zero')

        # ---- RULE 2: Abnormal rPPG morphology ----
        if (kurt > 10 or kurt < 0.5) and (hrv_rmssd > 200 or hrv_sdnns > 200):
            return ('FAKE', 1.0, 0.0,
                    'Abnormal rPPG morphology \u2014 spikey or flat pulse')

        # ---- RULE 3: Low SNR + inflated HRV ----
        if snr < -2.0 and (hrv_rmssd > 200 or hrv_sdnns > 200):
            return ('FAKE', 1.0, 0.0,
                    'Low rPPG signal quality \u2014 noise masquerading as pulse')

        # ---- RULE 4: No rPPG data -> cap at 0.70 ----
        prob_real = 1.0 - (0.6 * visual_score + 0.4 * ...)
        if bpm == 0 and hrv_rmssd == 0:
            prob_real = min(prob_real, 0.70)
            return self._threshold(prob_real,
                                    'No physiological confirmation \u2014 insufficient evidence')

        # ---- RULE 5: Normal weighted fusion ----
        prob_real = 1.0 - (0.6 * visual_score + 0.4 * ...)

        # Pulse override: valid pulse -> minimum 0.85
        if 50 <= bpm <= 120 and snr > 0:
            prob_real = max(prob_real, self.pulse_override)

        return self._threshold(prob_real, 'Standard weighted fusion')
```

7.1 Priority Order (Strict)

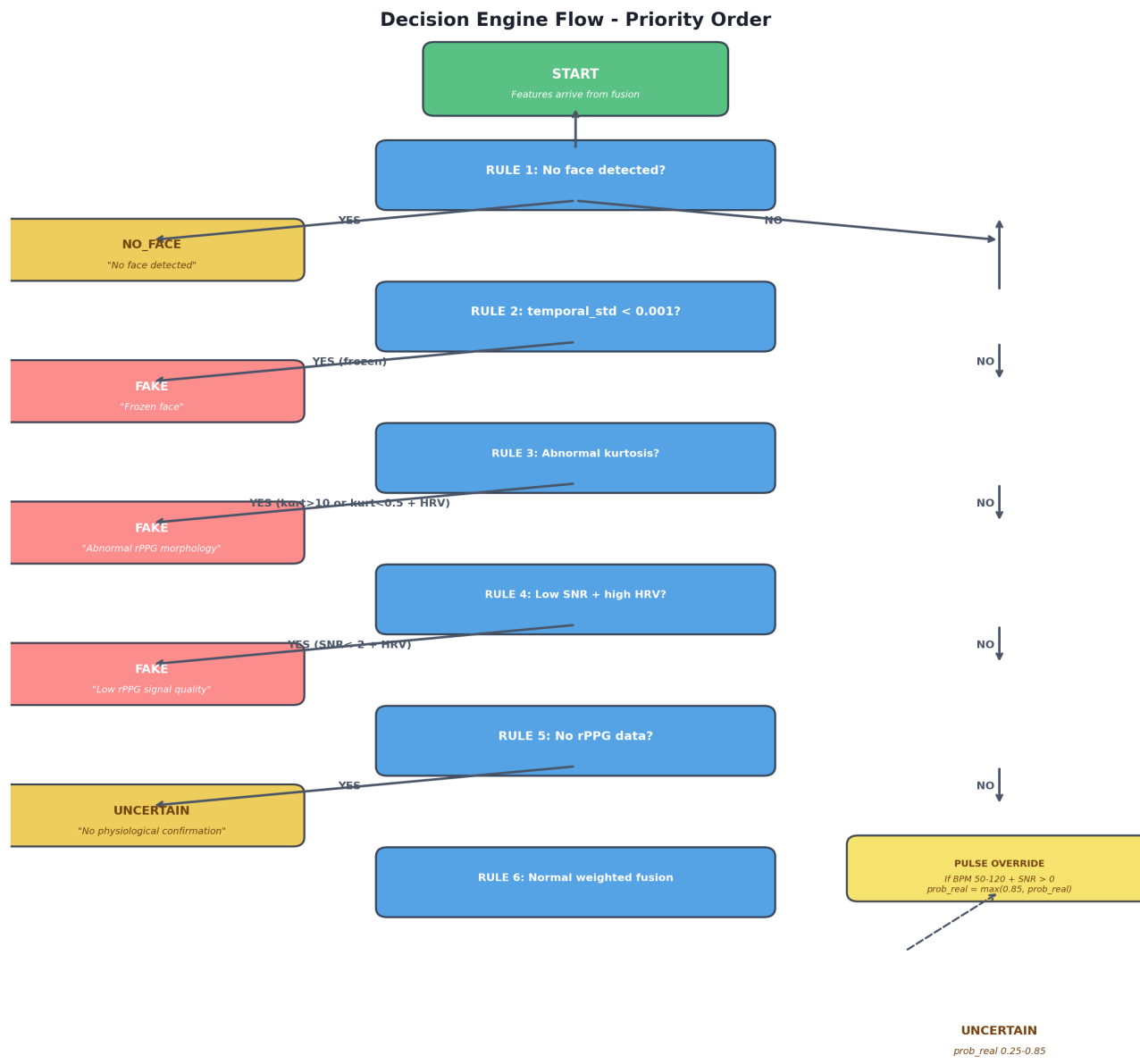
Priority	Rule	Condition	Trigger
1	Frozen face	temporal_std < 0.001	Physically impossible for real video
2	Abnormal morphology	kurt > 10 or kurt < 0.5 + HRV > 200	Deepfake pulse is spikey or flat noise
3	Low SNR + HRV	SNR < -2.0 + HRV > 200	Noise masquerading as pulse signal
4	No rPPG data	bpm == 0 and hrv_rmssd == 0	Cap prob_real at 0.70

5	Normal fusion	Default path	0.6 visual + 0.4 rPPG
5a	Pulse override	50 <= bpm <= 120 and SNR > 0	Valid pulse -> min 0.85 prob_real

Important: Rules 1-3 return immediately and **do not** enter the normal fusion path. The pulse override (rule 5a) **only** applies within the normal fusion path, ensuring deterministic FAKE decisions cannot be overridden by a false-positive pulse detection.

8. Decision Flowchart

The diagram below illustrates the decision engine's priority-ordered rules. Each diamond represents a deterministic check; if the condition matches, the system exits immediately with the corresponding verdict.



Threshold Configuration

Parameter	Value	Effect
real_threshold	0.85	prob_real >= 0.85 -> REAL
fake_threshold	0.25	prob_real <= 0.25 -> FAKE
pulse_override	0.85	Valid pulse -> minimum prob_real = 0.85
frame_sample_rate	3	Frames per second extracted from video

max_frames	90	Maximum frames to process (30s at 3 fps)
------------	----	--

9. API Layer (FastAPI Server)

The API is built with FastAPI and runs on Uvicorn. It exposes three endpoints and manages an in-memory result cache with a 10-minute TTL.

```
app = FastAPI(title="Deepfake Detector API", version="3.0.0")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
    expose_headers=["Content-Disposition"],
)

# In-memory cache: {report_id: (result_dict, xai_dict, timestamp)}
_result_cache = {}
_CACHE_TTL = 600 # 10 minutes

@app.get("/")
def root():
    return {"app": "Deepfake Detector", "version": "3.0.0", "status": "ready"}

@app.post("/detect")
async def detect(file: UploadFile = File(...)):
    video_bytes = await file.read()
    result, xai_data = analyze_video_bytes(video_bytes, file.filename)
    report_id = str(uuid.uuid4())[:8]
    _clean_cache()
    _result_cache[report_id] = (result, xai_data, time.time())
    return JSONResponse(**result, "report_id": report_id)

@app.get("/report/{report_id}")
async def report(report_id: str):
    entry = _result_cache.get(report_id)
    if entry is None:
        raise HTTPException(404, "Report not found or expired.")
    result, xai_data, _ = entry
    pdf_bytes = generate_pdf(result, xai_data=xai_data)
    return Response(content=pdf_bytes, media_type="application/pdf",
                    headers={"Content-Disposition": "attachment; filename=..."})
```

9.1 Cache Design

- **Structure:** Dictionary keyed by 8-character hex `report_id`
- **Value:** Tuple of (`result: dict`, `xai_data: dict`, `timestamp: float`)
- **Eviction:** Cleaned on every POST `/detect` call (lazy expiry)
- **TTL:** 600 seconds (10 minutes). After expiry, GET `/report/{id}` returns 404

9.2 CORS Configuration

CORS is wide open for development: `allow_origins=["*"]`, `allow_methods=["*"]`, `allow_headers=["*"]`. The `Content-Disposition` header is exposed to allow the browser to detect the PDF download filename. Credentialed requests are not supported; if needed, replace `["*"]` with specific origins.

10. PDF Report Generation (report.py)

The PDF report is generated using **ReportLab**, a professional-grade PDF library. The `generate_pdf(result, xai_data=None)` function produces a multi-section forensic report as a bytes object ready for HTTP response.

- **Decision Badge:** A colored box at the top showing REAL (green), FAKE (red), UNCERTAIN (amber), or NO_FACE (gray), rendered using ReportLab's Paragraph with HTML styling
- **Video Info Section:** Filename, duration estimate, frame count
- **Visual Stream Table:** ResNet-18 score, ViT-B/16 score, temporal variance, with reference baselines in gray text and color-coded health indicators (green = normal, yellow = borderline, red = suspicious)
- **rPPG Metrics Table:** BPM, HRV (RMSSD), HRV (SDNN), SNR, skewness, kurtosis. Each metric has a color-coded assessment in the Status column
- **rPPG Waveform Chart:** A matplotlib-generated PNG showing the filtered BVP signal with detected pulse peaks marked as red dots, embedded in the PDF
- **Fusion Features Table:** All 9 normalized features displayed with color highlights (red for suspicious values, yellow for borderline)
- **Face Crop Thumbnails:** Up to 3 sample face crops from the video (early, middle, late frames), encoded as base64 PNG and embedded in the PDF
- **Pipeline Metadata Footer:** Model versions, feature normalization info, inference timestamp
- **Metrics Reference Glossary:** Plain-English explanations of every metric in the report, making the report accessible to non-technical stakeholders (law enforcement, journalists, compliance teams)

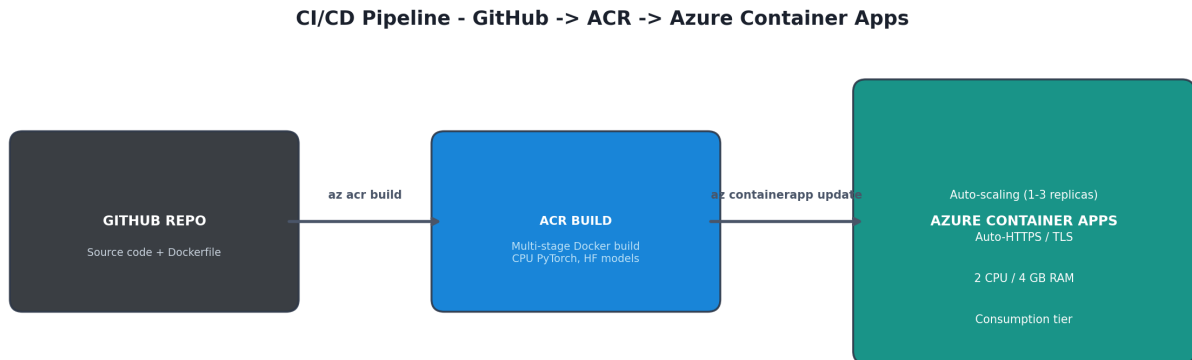
Report Generation Flow:

```
def generate_pdf(result, xai_data=None):
    doc = SimpleDocTemplate(buffer, pagesize=A4)
    story = []
    # 1. Decision badge (color-coded Paragraph)
    # 2. Video info table
    # 3. Visual stream metrics table
    # 4. rPPG metrics table with health assessments
    # 5. rPPG waveform chart (matplotlib -> BytesIO -> Image)
    # 6. Face crop thumbnails
    # 7. Fusion features table
    # 8. Metadata footer
    # 9. Metrics reference glossary (appendix)
    doc.build(story)
    return buffer.getvalue()
```

The report generation is **entirely server-side**. No external API or template engine is used. The report is generated on-the-fly and returned as a byte stream.

11. CI/CD & Deployment Pipeline

The system is deployed on Azure Container Apps using a multi-stage Docker build pipeline. The deployment is triggered manually via Azure CLI commands.



11.1 Docker Build (Dockerfile)

```
# Stage 1: Builder
FROM python:3.10-slim-bookworm AS builder

# Install OpenCV dependencies (Debian 12 bookworm)
RUN apt-get update && apt-get install -y libgl1-mesa-glx libglu1-mesa-dev libsm6 libxext6 libxrender1

WORKDIR /app
COPY requirements.txt .

# CPU-only PyTorch (install FIRST to avoid --index-url override)
RUN pip install --no-cache-dir --index-url https://download.pytorch.org/whl/cpu \
    torch torchvision

# Remaining packages from PyPI
RUN pip install --no-cache-dir -r requirements.txt

# Download pretrained models at build time
COPY download_models.py .
RUN python download_models.py

# Stage 2: Runtime
FROM python:3.10-slim-bookworm
RUN apt-get update && apt-get install -y libgl1-mesa-glx libglu1-mesa-dev \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.10/site-packages /usr/local/lib/python3.10/site-packages
COPY --from=builder /app /app
COPY . /app
RUN useradd -m appuser && chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

11.2 Deployment Commands

```
# 1. Build from GitHub source
az acr build --registry deepfakeprojectacr2026 \
    --image video-deepfake-detector:latest \
```

<https://github.com/Alagapie/Video-DeepFake-Detector.git>

2. Create Container Apps environment (one-time)

```
az containerapp env create --name detector-env \  
  --resource-group audio-deepfake-rg --location francecentral
```

3. Deploy the app

```
az containerapp create --name deepfake-api \  
  --resource-group audio-deepfake-rg \  
  --environment detector-env \  
  --image deepfakeprojectacr2026.azurecr.io/video-deepfake-detector:latest \  
  --registry-server deepfakeprojectacr2026.azurecr.io \  
  --target-port 8000 --ingress external \  
  --min-replicas 1 --max-replicas 3 \  
  --cpu 2.0 --memory 4Gi
```

4. Update (new build)

```
az containerapp update --name deepfake-api \  
  --resource-group audio-deepfake-rg \  
  --image deepfakeprojectacr2026.azurecr.io/video-deepfake-detector:latest \  
  --revision-suffix fix1
```

12. Configuration Reference (config.yaml)

```
inference:
  frame_sample_rate: 3      # Frames per second to extract
  max_frames: 90           # Cap at 30 seconds (90 / 3)
  face_confidence: 0.5     # Face detection threshold
  real_threshold: 0.85     # prob_real >= this -> REAL
  fake_threshold: 0.25     # prob_real <= this -> FAKE
  pulse_override: 0.85     # Valid pulse -> min prob_real
  weights_dir: weights     # Path to model weight files

rppg:
  bandpass_low: 0.8        # Hz - lower bound for pulse filter
  bandpass_high: 3.0       # Hz - upper bound for pulse filter
  fps: 3                   # rPPG processing frame rate
```

Parameter Tuning Guidelines:

default	Effect of Increasing	Effect of Decreasing
	More frames = slower, better rPPG	Faster, less rPPG accuracy
	Longer videos processed	Shorter videos, faster response
s	Fewer false positives	More REAL verdicts
s	More FAKE verdicts	Fewer false positives for FAKE
s	Stronger pulse confirmation	Weaker pulse influence
	Filters out lower pulse rates	Captures slower heart rates
	Captures higher pulse rates	More aggressive filtering

13. File-by-File Code Map

Complete listing of every Python file in the project with its role and line count.

File	Lines	Role
api.py	158	FastAPI server: POST /detect, GET /report/{id}, CORS, result cache
detect.py	~100	CLI entry point: python detect.py video.mp4 [--json]
report.py	~400	PDF report generator (ReportLab): badge, tables, charts, glossary
download_models.py	~30	Downloads ResNet-18 + ViT-B/16 from HuggingFace at Docker build time
config.yaml	15	Central configuration: thresholds, frame rate, rPPG params
requirements.txt	17	Dependencies: torch, torchvision, opencv, fastapi, reportlab, neurokit2
streams/face_detector.py	~80	FaceDetector class: OpenCV Haar cascade + MediaPipe fallback
streams/visual_stream.py	~60	VisualStream class: ResNet-18 + ViT-B/16 wrapper
streams/rppg_stream.py	~120	RPPGExtractor: CHROM/POS, bandpass filter, peak detection
utils/video_io.py	~50	extract_frames(): VideoCapture, frame sampling, preprocess_face()
utils/signal_proc.py	~70	butter_bandpass, find_peaks wrapper, compute_hrv with signal return
fusion/build_features.py	~40	9-dim feature vector assembly + MinMaxScaler
fusion/decision.py	~100	FusionEngine: weighted fusion + 5 override rules
Dockerfile	~40	Multi-stage build: slim-bookworm, CPU PyTorch, HF model download
.dockerignore	13	Excludes venv, weights, mp4, pdf, git from Docker context
.gitignore	15	Excludes venv, pycache, mp4, pdf from git tracking

14. Dependency Graph

The import dependency structure of the project. Arrows mean 'imports from'.

```
api.py
+-- utils/video_io.py      (extract_frames, preprocess_face)
+-- streams/face_detector.py (FaceDetector)
+-- streams/visual_stream.py (VisualStream)
+-- streams/rppg_stream.py  (RPPGExtractor)
+-- fusion/build_features.py (build_feature_vector, FEATURE_NAMES)
+-- fusion/decision.py      (FusionEngine)
+-- report.py               (generate_pdf)
+-- config.yaml

visual_stream.py
+-- torch, torchvision
+-- utils/video_io.py      (preprocess_face)

rppg_stream.py
+-- neurokit2              (CHROM, POS)
+-- scipy.signal           (butter, sosfiltfilt, find_peaks)
+-- scipy.stats            (kurtosis, skew)
+-- numpy

decision.py
+-- numpy
+-- config.yaml

report.py
+-- reportlab              (SimpleDocTemplate, Table, Paragraph, Image)
+-- matplotlib             (figure, plot, savefig to BytesIO)
+-- io (BytesIO)
+-- base64
```

External Dependencies:

Package	Version	License	Purpose
torch	2.x	BSD	Deep learning inference (CPU)
torchvision	0.x	BSD	Image preprocessing & transforms
opencv-python	4.x	Apache 2.0	Video I/O, face detection, image processing
fastapi	0.x	MIT	REST API framework
uvicorn	0.x	BSD	ASGI server
reportlab	4.x	BSD	PDF report generation
matplotlib	3.x	BSD	rPPG waveform chart rendering
neurokit2	0.x	MIT	rPPG (CHROM, POS) algorithms
numpy	1.x	BSD	Numerical computation
scipy	1.x	BSD	Signal filtering, peak detection, statistics
scikit-learn	1.x	BSD	MinMaxScaler for feature normalization
pyyaml	6.x	MIT	Config file parsing
huggingface-hub	0.x	Apache 2.0	Model weight download

